

Programrendszerek fejlesztése

Modern frontend alkalmazások

Tananyag-összefoglaló – 2026. tavasz

Receptgyűjtemény – MEAN Stack demonstrációs projekt

MongoDB · Express.js · Angular · Node.js

1. Bevezetés – Hol tartunk most?

A félév során eddig felépítettünk egy működő backend rendszert: egy Express.js alapú REST API-t, amely MongoDB adatbázissal kommunikál, és támogatja a JWT alapú hitelesítést és jogosultságkezelést. Ez a szerver képes CRUD (Create-Read-Update-Delete) műveleteket végezni, de jelenleg csak Postman-nel vagy cURL-lel tudjuk elérni.

Ez nyilvánvalóan nem elegendő egy valós alkalmazáshoz: a felhasználók nem JSON válaszokat akarnak olvasni, hanem vizuális felületen szeretnének navigálni, űrlapokat kitölteni és eredményeket látni. Szükségünk van egy kliens oldali (frontend) alkalmazásra.

Párhuzam: a backend a motor, a frontend a műszerfal és a kormány. A motor nélkül nem megy az autó, de a kormány nélkül senki nem tudja vezetni.

2. A klasszikus webtől a modern SPA-ig

2.1. Hagyományos szerver-renderelt modell

A web kezdeti korszakában minden interakció egy teljes kérés-válasz ciklust jelentett. A felhasználó rákattintott egy linkre, a böngésző küldött egy HTTP kérést a szervernek, a szerver pedig egy komplett HTML oldalt renderelt és küldött vissza. Minden kattintásnál az egész oldal újratöltődött: a fejléc, a menü, a lábléc, minden – még akkor is, ha csak egy apró adat változott.

Tipikus példák: PHP, JSP, ASP alapú alkalmazások, ahol a szerver oldalon sablonok (template-ek) voltak, amelyekbe a szerver beillesztette az adatbázisból kiolvasott adatokat.

Előnyök: egyszerű gondolkodásmód, a szerver teljes kontrollt gyakorol a megjelenítés felett, a SEO természetesen működik (a keresőrobotok kész HTML-t kapnak).

Hátrányok: minden navigáció villanásnyi fehér képernyőt jelent, a szerver terhelése magasabb (minden kérésre teljes HTML-t kell generálnia).

2.2. Az AJAX áttörés

A fordulópontot az AJAX (Asynchronous JavaScript and XML) jelentette, nagyjából 2005 környékén. A Gmail és a Google Maps megmutatta, hogy a böngésző képes a háttérben, oldalújratöltés nélkül kommunikálni a szerverrel. Így lehetővé vált az oldal egy részének frissítése a teljes újratöltés nélkül.

Az AJAX önmagában még nem oldotta meg a teljes architektúrát. Jellemzően jQuery-vel írt szkriptek módosították a DOM-ot, ami nagyobb alkalmazásoknál gyorsan átláthatatlan, nehezen karbantartható kóddá vált.

2.3. Single Page Application (SPA)

Az SPA az AJAX gondolatmenetét viszi végig a logikus végkifejletéig. Az alapötlet: a böngésző egyetlen HTML oldalt tölt be, és utána a JavaScript alkalmazás veszi át az

irányítást teljes egészében. A navigáció, az oldalak közötti váltás, az adatok megjelenítése mind a kliens oldalon történik. A szerver ettől kezdve csak nyers adatot (JSON-t) szolgáltat egy API-n keresztül.

Ez egy alapvető **felelősség-áthelyezés**: a megjelenítés (rendering) a szerverről a kliensre költözött. A szerver „butább” lett (csak adat), a kliens „okosabb” (megjelenítés + navigáció + állapotkezelés).

Ez pontosan az az architektúra, amit a projektünkben megvalósítunk: Express szerver (REST API) + Angular kliens (SPA).

3. SPA előnyök és hátrányok

Az SPA nem ezüstgolyó, hanem egy architekturális döntés, aminek vannak trade-off-jai. Egy jó mérnök nem azt kérdezi, hogy „mi a legújabb”, hanem azt, hogy „mi illik a problémához”.

3.1. Előnyök

Gyorsabb felhasználói élmény

Az első betöltés után az alkalmazás nem kér a szervertől teljes HTML oldalakat, csak adatot. Az oldalváltás szinte azonnali, mert a kliens már rendelkezik az alkalmazás kódjával, csak az adatokat kell lecserélni. Nem villan fehérre a képernyő, nincsenek teljes újratöltések.

Frontend és backend tiszta szétválasztása

Ha a szerver csak REST API-t biztosít, akkor ugyanahhoz a backend-hez bármilyen kliens csatlakozhat: webes Angular alkalmazás, mobil app, egy másik csapat által fejlesztett React frontend, vagy akár egy IoT eszköz. A két rendszer között mindössze a REST API a szerződés.

Gazdagabb interakciók

Mivel az alkalmazás logikája a böngészőben fut, könnyebb komplex interakciókat megvalósítani: drag-and-drop, valós idejű szűrés, animációk, azonnali validáció űrlapokon.

Csökkentett szerver terhelés

A szerver nem foglalkozik HTML renderelésével, csak adatokat szolgáltat. A JSON válaszok jellemzően kisebbek, mint egy komplett HTML oldal.

3.2. Hátrányok

Hosszabb kezdeti betöltési idő

Az első látogatáskor a böngészőnek le kell töltenie a teljes JavaScript alkalmazást (a framework-öt, a komponenseket, a routing logikát) mielőtt bármi megjelenne. Egy hagyományos oldalnál az első válasz már kész HTML, ami azonnal megjeleníthető.

SEO kihívások

A keresőmotorok hagyományosan úgy működtek, hogy letöltötték a HTML-t és indexelték a tartalmát. Egy SPA esetében a kezdeti HTML gyakorlatilag üres (csak egy `<app-root>` tag), és a tényleges tartalom csak a JavaScript lefutása után jelenik meg. Belső vállalati alkalmazásoknál (mint a Receptgyűjtemény) ez irreleváns, de publikus weboldalaknál komoly szempont.

Bonyolultabb állapotkezelés

Hagyományos alkalmazásban az állapot a szerveren van. SPA-nál az állapot nagy része a kliensre kerül: be van-e jelentkezve a felhasználó, milyen szűrők aktívak, milyen adatok töltődtek be. Ezt mind a JavaScript alkalmazásnak kell nyomon követnie.

JavaScript függőség

Ha a böngészőben ki van kapcsolva a JavaScript, az SPA egyáltalán nem működik. Ha a JS bundle betöltése közben hiba történik, a felhasználó üres oldalt lát.

SPA Előnyök	SPA Hátrányok
Gyors, gördülékeny felhasználói élmény	Hosszabb kezdeti betöltési idő
Frontend-backend szétválasztás	SEO kihívások
Gazdagabb interakciók	Bonyolultabb állapotkezelés
Csökkentett szerver terhelés	JavaScript függőség

4. Frontend keretrendszerek összehasonlítása

Három meghatározó keretrendszer/könyvtár uralja a modern frontend fejlesztést. Fontos megérteni a különbségeiket, hogy a technológiaválasztást mérnöki döntésként kezeljük, ne divatkövetésként.

4.1. React (Meta, 2013)

A React szigorú értelemben **nem keretrendszer, hanem UI könyvtár**. Kizárólag a nézet (view) réteggel foglalkozik: komponensek létrehozása és renderelése. Minden mást – routing, állapotkezelés, HTTP kommunikáció – külső könyvtárakból kell összeválogatni (React Router, Redux/Zustand, Axios stb.).

JSX szintaxist használ, ami HTML-szerű kódot ágyaz a JavaScript-be. Az ökoszisztémája a legnagyobb, a munkaerőpiaci kereslet globálisan a legmagasabb.

4.2. Angular (Google, 2016)

Fontos: az AngularJS (2010) és az Angular (2016+) két teljesen különböző rendszer. Az Angular egy a nulláról TypeScript-ben újraírt, modern keretrendszer.

Az Angular „**batteries-included**” (véleményezett, opinionated) keretrendszer: szinte mindenre van beépített megoldás – routing, HTTP kliens, űrlapkezelés, dependency injection, tesztelés, animációk, i18n. Nem kell külső könyvtárakat válogatni.

TypeScript-first megközelítés: nem opcionális, hanem alapértelmezett és elvárt. A tanulási görbéje a három közül a legmeredekebb, mert egyszerre kell megérteni a TypeScript-et, a komponensmodellt, a dependency injection-t és az RxJS-t.

4.3. Vue.js (Evan You, 2014)

A Vue-t gyakran az Angular és a React közötti „arany középutként” pozicionálják. Progresszív keretrendszer: elkezdheted nagyon kicsiben, és fokozatosan építheted ki teljes SPA-vá.

Single File Component (SFC) modellt használ: egyetlen .vue fájlban van a sablon, a logika és a stílus. A tanulási görbéje a leglaposabb, a dokumentációja kiválóan jó.

4.4. Összehasonlító táblázat

Szempont	React	Angular	Vue
Típus	UI könyvtár	Teljes keretrendszer	Progresszív keretrendszer
Tanulási görbe	Közepes	Merdek	Lapos
TypeScript	Opcionális	Kötelező	Opcionális (erős)
Véleményezettség	Alacsony (szabad)	Magas (strukturált)	Közepes

Piaci kereslet	Legmagasabb	Erős (vállalati)	Növekvő
Háttér	Meta	Google	Közösségi

4.5. Miért Angular-t használunk a kurzusban?

- A MEAN stack természetes része (MongoDB, Express, Angular, Node.js) – a teljes stack JavaScript/TypeScript alapú.
- A véleményezetség oktatási előny: nem kell architekturális döntésekkel birkózni, a struktúra adott.
- A TypeScript megtanítja értékelni a statikus típusosságot és a fordítási idejű hibajelzést.
- **Tanulástranszfer:** aki megérti az Angular mögötti koncepciókat, React-et vagy Vue-t is gyorsabban veszi fel, mert a fogalmak ugyanazok.

5. Az Angular keretrendszer alapjai

5.1. Az Angular filozófiája

Az Angular egy „batteries-included” keretrendszer, amelynek alapelvei:

- **Komponens-alapú architektúra:** minden UI elem egy komponens
- **TypeScript-first:** statikus típusosság alapértelmezetten
- **Deklaratív templating:** a sablon leírja, MIT jelenítsen meg, nem HOGYan
- **Dependency Injection:** a szolgáltatások automatikus beinjektálása
- **Angular CLI:** központi fejlesztőeszköz, ami generál, buildelhet, tesztelhet

5.2. Komponens-alapú gondolkodás

A komponens egy **önálló, újrafelhasználható UI építőelem**, amelynek saját sablonja (HTML), stílusa (CSS) és logikája (TypeScript) van. A komponensek fa struktúrában épülnek egymásra (component tree).

A Receptgyűjtemény alkalmazásunkban például a következő komponensekre bontható egy oldal:

- AppComponent – a gyökér komponens
 - HeaderComponent – navigációs sáv
 - RecipeListComponent – receptek listája
 - RecipeCardComponent – egyetlen receptkártya
 - FooterComponent – lábléc

Nem oldalakat tervezünk, hanem építőköveket, amelyekből oldalak állnak össze.

5.3. Angular projekt felépítése

Egy Angular projekt kulcs elemei:

Fájl / Mappa	Leírás
angular.json	Központi projektkonfiguráció
src/main.ts	Az alkalmazás belépési pontja
src/app/	Fő munkaterület: komponensek, szolgáltatások, modulok
app.routes.ts	Routing konfiguráció (URL ↔ komponens hozzárendelés)
app.config.ts	Alkalmazás-szintű konfiguráció és provider-ek

5.4. Adatkötés (Data Binding)

Az adatkötés a sablon és a komponens logikája közötti kommunikáció alapja. Az Angular négyféle adatkötést különböztet meg:

Típus	Szintaxis	Példa
Interpoláció	{{ kifejezés }}	{{ recipe.title }}
Property binding	[property]	[value]="recipe.id"
Event binding	(event)	(click)="save()"
Two-way binding	[(ngModel)]	[(ngModel)]="searchText"

Az **interpoláció** a leggyakoribb: adat megjelenítése a sablonban. A **property binding** adatot ad át a DOM-nak. Az **event binding** felhasználói eseményeket kezel. A **two-way binding** szinkronizál mindkét irányba (pl. űrlapmezők).

5.5. Routing: navigáció oldalak között

Bár SPA-ról beszélünk, a felhasználó mégis „oldalak” között navigál – ezt a **Router modul** teszi lehetővé. URL-eket rendelünk komponensekhez, a `<router-outlet>` jelöli ki a dinamikus tartalom helyét, és a `routerLink` direktívával hozunk létre navigációs linkeket.

A Receptgyűjtemény alkalmazásunkban a következő route-ok vannak definiálva:

Útvonal	Betöltött modul / Komponens
/auth/login	Bejelentkezés
/auth/register	Regisztráció
/recipes	Receptlista
/recipes/:id	Recept részletek
/admin	Admin felület (lazy loaded)
/categories	Kategória kezelés (lazy loaded)

A **lazy loading** (lusta betöltés) azt jelenti, hogy egy-egy modul kódja csak akkor töltődik le, amikor a felhasználó először navigál az adott útvonalra. Ez csökkenti a kezdeti betöltési időt.

6. A Receptgyűjtemény alkalmazás architektúrája

6.1. Adatmodell

Az alkalmazás in-memory (memóriában tárolt) mock adatokkal dolgozik. Később ezeket az adatokat a backend REST API-ból fogjuk lekérni. Az adatmodell öt entitásból áll:

Entitás	Kulcs mezők	Kapcsolat
User	id, username, email, role	Recepteket hoz létre, értékeléseket ír
Category	id, name, description	Receptek csoportosítása
Recipe	id, title, description, categoryId	Tartozik egy kategóriához
Ingredient	name, quantity	Beágyazva a Recipe-ben
Rating	id, userId, recipeId, score	Összeköti a User-t és a Recipe-t

6.2. Modulok és komponensek

Az alkalmazás feature modulokra van bontva, mindegyik lazy loading-gal töltődik:

- **AuthModule:** Login, Register
- **RecipesModule:** RecipeList, RecipeDetail, RecipeForm, RecipeSearch
- **CategoriesModule:** CategoryList, CategoryForm, CategoryFilter
- **AdminModule:** AdminDashboard, CategoryManagement, RecipeManagement, RatingModeration
- **UserModule:** Profile, MyRatings
- **Shared:** Header, Footer, RecipeCard, RatingStars, ConfirmDialog, CategoryChip

6.3. Hogyan fog ez összekapcsolódni a backend-del?

Jelenleg az alkalmazás in-memory adatokkal dolgozik (a `src/app/in-memory/` mappában definiált tömbök). A következő lépés az **Angular szolgáltatások (services)** bevezetése, amelyek a `HttpClient` segítségével REST API hívásokat indítanak a backend felé.

Az architektúra sémája:

Angular App ↔ HTTP (REST) ↔ Express API ↔ MongoDB

7. Kulcsfogalmak összefoglalása

Az alábbi fogalmak ismerete elengedhetetlen a tananyag megértéséhez:

Fogalom	Definíció
SPA	Single Page Application – egyetlen HTML oldal, a kliens kezeli a navigációt és a renderelést.
Komponens	Önálló, újrafelhasználható UI építőelem saját sablonnal, stílussal és logikával.
Routing	URL-ek hozzárendelése komponensekhez, lehetővé téve az „oldalak” közötti navigációt SPA-ban.
Lazy Loading	Modulok betöltése csak akkor, amikor a felhasználó először navigál az adott útvonalra.
Data Binding	A komponens logikája és a sablon közötti adatszinkronizáció mechanizmusa.
Interpoláció	{{ kifejezés }} szintaxis adat megjelenítésére a sablonban.
Direktíva	Angular utasítások a sablonban, pl. *ngFor (iteráció), *ngIf (feltételes megjelenítés).
Dependency Injection	Szolgáltatások automatikus biztosítása a komponensek számára a keretrendszer által.
MEAN Stack	MongoDB + Express.js + Angular + Node.js – teljes JavaScript/TypeScript alapú web stack.
CRUD	Create-Read-Update-Delete – az alapvető adatműveletek összefoglaló neve.
REST API	Reprezentációs állapotátvitel – szabványos HTTP módszerekkel (GET, POST, PUT, DELETE) elérhető végpontok.
JWT	JSON Web Token – a felhasználó hitelesítésére szolgáló, aláírt token.